Mohar

Roadmap, flow maps, seal-mismatch theory,
key division S1–S4, and the hardware layer

A study companion to the main guide. This document answers four questions: what order the system gets built in, how a package actually flows from sealing to destruction, what a seal mismatch really tells you, and how the content key is split four ways so that no single person — including us — can open a paper early.

Part F adds the hardware layer: the ESP32 room monitor, the electronic seal lock, and the new witness station carrying a camera and a fingerprint sensor.

COMPANION TO

Mohar — Study Guide (basics to advanced)

COVERS

Roadmap · end-to-end flow · seal mismatch · Shamir S1–S4 · ESP32 hardware

STATUS MARKS

BUILT = running code · PARTIAL = some of it exists · PLANNED = designed, not written

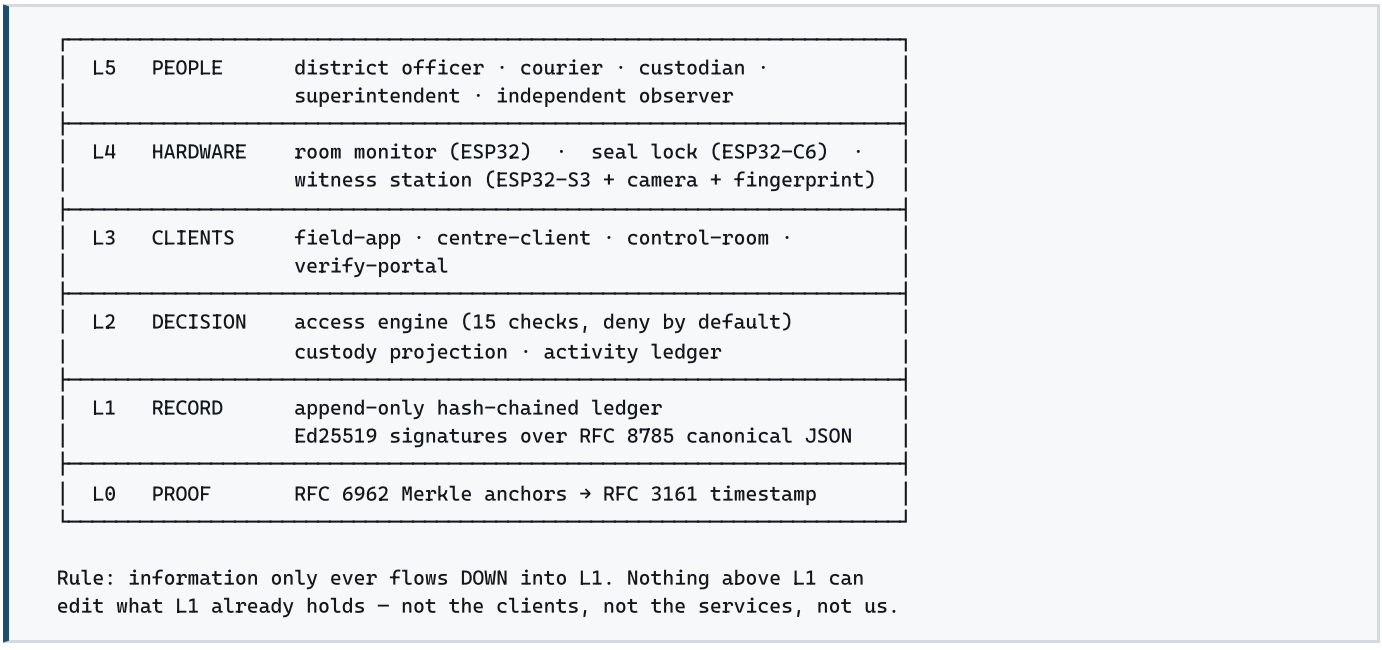
A The basis — what the whole project rests on

Before any roadmap makes sense, four ideas have to be fixed. Everything else in Mohar is a consequence of these.

A.1 The four load-bearing ideas

#	Idea	Consequence throughout the system
1	**The failure is evidentiary, not detective.** Leaks get detected; cases collapse.	We optimise for evidence that survives cross-examination, not for alarm speed. This is why we store "412 m" and not "outside_geofence".
2	**Absence must be as loud as presence.** A missing record is a finding.	State is derived by replaying events, so a hole in the record produces a visible gap rather than a plausible timeline. Heartbeats make silence detectable in hardware too.
3	**Controls must fail closed.** Broken infrastructure must deny, never grant.	Keys expire by arithmetic with no cron job. A dead beacon means no S2. A dead room monitor raises an event rather than going quiet.
4	**No single point of trust — including us.**	The content key is split 3-of-4 across three organisations plus physics. Append-only is enforced by database grants, not our own code.

A.2 The layer diagram



Rule: information only ever flows DOWN into L1. Nothing above L1 can edit what L1 already holds — not the clients, not the services, not us.

A.3 Where cryptography stops

> At the printer tray. Past that point it is paper, a room, and people with phones. Everything after that is physical control and attribution, which is exactly what the hardware layer in Part F is for — and even that is **tamper-evident, not tamper-proof**. Say this before a judge says it to you.

B The roadmap

BUILT running code PARTIAL partly exists PLANNED designed, not written

Phase 0 — Foundation BUILT

Ledger core	Append-only event table, hash chain `SHA256(prevHash ‖ bodyHash)`, genesis = 32 zero bytes
Canonical JSON	RFC 8785 JCS via `canonicalize` in `packages/crypto-core`
Signatures	Ed25519 over `bodyHash`; device enrolment and revocation
Append-only enforcement	Postgres grants — `mohar_app` holds SELECT + INSERT only
Custody projection	`domain/custody.ts` — state replayed from events, 7 anomaly detectors
Access engine	`domain/policy.ts` — 15 checks, deny by default, never short-circuits
Custody keys	6-hour epochs, Crockford base32, SHA-256 stored, expiry by arithmetic
Control room	Overview · Workflow timeline · Packages · Activity · Keys · Devices · Integrity
Shamir primitive	`packages/crypto-core/src/shamir.ts` — 3-of-4 with per-share commitments

Phase 1 — Verification surface PARTIAL

Merkle anchors	**Built.** Daily RFC 6962 tree, root stored with first/last seq and tree size
Chain verification API	**Built.** `/verify/chain` recomputes and reports breaks
RFC 3161 timestamping	**Missing.** The `notarised` flag is never set true
verify-portal	**Stub.** Third-party inclusion-proof checking not exposed yet

> This is the highest-value gap to close for credibility. Without external timestamping, the anchor proves nothing against an attacker who owns the database — it is a fixed point that only exists *inside* the thing it is supposed to check.

Phase 2 — Crypto completion PLANNED

- Wire the existing Shamir primitive into the live unlock path (Part E).
- `tlock` encryption of S2 to a future drand beacon round.
- WebAuthn platform authenticators behind S3 and S4.
- XChaCha20-Poly1305 bundle encryption under content key `K`.

Phase 3 — Room monitor PLANNED

Existing spec, `firmware/room-monitor`. Door reed switch, mmWave presence, dual ToF footfall, light, DS3231 RTC, microSD buffer, 30-second heartbeat, per-device HMAC.

Phase 4 — Witness station NEW

ESP32-S3 with camera and fingerprint sensor, scoped to the unlock ceremony only. Full design in Part F. This is the phase that adds biometric attribution and visual evidence at the single highest-value moment in the whole chain.

Phase 5 — Seal lock PLANNED

ESP32-C6, `firmware/seal-lock`. Latch control, continuous tamper loop, monotonic replay counter, pinned authority public key, tamper events written to flash before any radio transmission.

Phase 6 — Hardening PLANNED

- Authentication and authorisation at the `gateway` service — currently a stub, and the ledger API is unauthenticated.
- Rate limiting on `/access/request`.
- Device attestation actually verified rather than merely enrolled.
- **Git history.** The repository has zero commits; every revert so far has been a hand reconstruction.

Phase 7 — Pilot

One district, one examination, mock drill first. Publish the fallback-invocation count. "Fallback used at 3 of 4,750 centres" is credible; hiding the path is not.

What to actually finish before the presentation

> **Do not start Phase 5 or 6.** For a hackathon the winning move is depth over breadth: Phase 0 is genuinely strong, Phase 4 gives you something physical on the table, and closing the Phase 1 timestamping gap converts your biggest honest weakness into a strength. Everything else is a slide, not a sprint.

C Flow map — a package from sealing to destruction

C.1 The eight stages as a pipeline

```
 S1        S2          S3         S4          S5        S6       S7       S8
seal → dispatch → transit → custodian → centre → unlock → print → destroy
 |        |          |          |          |        |        |        |
 DO       DO       courier   custodian    supt     supt     supt     supt
 |        |          |          |          |        |        |        |
 └────────┴──────────┴──────────┴──────────┴────────┴────────┴────────┘
  every hop: custody key presented → 15 checks → signed event → chain

DO = district officer   supt = superintendent

NOTE: these stage numbers S1..S8 are the CUSTODY stages.
They are NOT the same as the key shares S1..S4 in Part E.
Different things, same letter. Do not mix them up in the demo.
```

C.2 What happens at every single hop

```
┌─ actor arrives at a hop ──────────────────────────────────────┐
│                                                               │
│  1. control room ISSUES a custody key for (package, stage)    │
│       key = MHR-<STAGE>-XXXX-XXXX-XXXX   (128 bits, base32)   │
│       stored as SHA-256 only; shown once; unique per epoch    │
│                                                               │
│  2. actor PRESENTS key on an enrolled device                  │
│       + device id  + person id  + GPS + accuracy              │
│       + seal serial read off the physical seal                │
│       + device clock                                          │
│                                                               │
│  3. ACCESS ENGINE runs all 15 checks — never short-circuits   │
│       key_presented · key_valid · key_in_window ·             │
│       key_stage_match · device_enrolled · device_binding ·    │
│       roster_membership · role_permitted · geofence ·         │
│       geo_accuracy · custody_window · clock_skew ·            │
│       seal_serial · package_state · exam_active               │
│                                                               │
│  4. ATTEMPT IS WRITTEN  ← before the caller is told anything  │
│       into led.access_attempt (SELECT + INSERT grants only)   │
│                                                               │
│  5. outcome returned:  granted  |  denied + evidence per check │
│                                                               │
│  6. if granted, actor's device SIGNS the custody event        │
│       bodyHash = SHA256(JCS(body))                            │
│       sig      = Ed25519(devicePrivKey, bodyHash)             │
│                                                               │
│  7. LEDGER APPENDS                                            │
│       hash = SHA256(prevHash ‖ bodyHash)                      │
│                                                               │
│  8. PROJECTION re-derives package state from all events       │
│       → new state, new holder, anomalies recomputed           │
└───────────────────────────────────────────────────────────────┘
```

C.3 Offline path

```
strongroom, no signal
      │
      ├── device signs event LOCALLY (private key never leaves device)
      ├── event queued with a client-generated UUID
      ├── room monitor writes to microSD BEFORE any radio attempt
      │
      └── signal returns ──► upload ──► ledger sees UUID
                                              │
                                   already present? ──► return existing
                                   new?             ──► append
                              (idempotent — retry is always safe)
```

C.4 The unlock ceremony (S6) in detail

```
T−0   drand beacon publishes round for startAt
        └──► S2 becomes recoverable — and not one second earlier

T+1   superintendent authenticates      → releases S3
T+2   independent observer authenticates → releases S4
        (two-person rule; two DISTINCT humans, two distinct credentials)

T+3   S1 unwrapped from authority passphrase (Argon2id)

T+4   any 3 of {S1,S2,S3,S4} combine IN MEMORY → content key K
        each share checked against its commitment FIRST
        → a bad share is NAMED, not silently absorbed

T+5   bundle decrypted (XChaCha20-Poly1305), rendered to spooler
        plaintext NEVER touches disk

T+6   printer meters exactly N copies, each watermarked → traceable

T+7   K zeroised in memory

T+8   KEY_DESTROYED signed and queued
        └──► if this never arrives, the projection raises
             key_not_destroyed and the act stays OPEN
```

C.5 Data flow into the ledger

```
 people ──┐
devices ──┤
   ESP32 ─┼──► signed / HMAC'd records ──► LEDGER (append-only)
 sensors ──┤                                  │
 cameras ──┘                                  ├──► projection ──► packages
                                              ├──► activity ledger
                                              ├──► chain verification
                                              └──► daily Merkle anchor
                                                       │
                                                       └──► RFC 3161  [gap]
```

D Theory of the seal mismatch

The seal serial is the cheapest and strongest physical signal in the entire system. Understand precisely what it does and does not prove.

D.1 What a seal actually is

A tamper-evident seal is a one-time closure carrying a unique printed serial. Its security property is **not** that it cannot be broken — it can, trivially. It is that it **cannot be broken and restored to its original state**. To open a sealed bundle you must destroy the seal, and to conceal that you must fit a new seal, which necessarily carries a *different* serial.

> **The whole theory in one line.** The attacker's problem is not opening the bundle. It is that reclosing it forces them to either present a serial that does not match the registry, or forge a seal bearing a serial they must first have learned. The seal converts a silent physical act into a recorded contradiction.

D.2 Registration and re-reading

```
S1 seal     ── serial MH-4471-A is REGISTERED against the package
               (written into a signed ledger event; now immutable)

S2..S8      ── at EVERY subsequent hop the actor reads the physical
               serial and submits it with the access request

               engine compares  read  vs  registered
                     │
                     ├── equal      → seal_serial PASSES
                     ├── different  → seal_serial_mismatch
                     └── absent     → seal_serial_not_read
```

D.3 Why "not read" is its own failure, not a pass

> This is the subtle part, and it is worth a full sentence in your demo. If a missing serial were treated as "nothing to compare, therefore no problem", then **the cheapest attack on the entire system would be to simply not look at the seal.** Silence would become the safest move for a guilty actor. So `seal_serial_not_read` is a distinct denial reason. Not looking is itself a finding.

Generalise it: everywhere in Mohar, a missing observation is treated as a negative observation, never as a neutral one. The custody gap works the same way, and so does the ESP32 heartbeat — unplugging the device does not produce silence, it produces an event.

D.4 Two layers of detection, and why both

	Access engine — `policy.ts`	Projection — `custody.ts`
When	Real time, during the request	Retrospective, replaying all events
Code	`seal_serial` check → `seal_serial_mismatch`	`seal_serial_changed` anomaly
Effect	**Blocks** the access	**Flags** the package for investigation
Sees	Only this one attempt	The entire history, including hops that never produced an attempt

The engine cannot catch a substitution that happens between two hops where nobody requested access. The projection can, because it compares every recorded reading against the registration across the whole life of the

package. One is a gate; the other is an auditor. You need both.

D.5 What a mismatch actually means — the inference chain

```
observed:  registered MH-4471-A, read MH-4471-B
     │
     ├─ the physical seal now on this bundle is not the one applied at S1
     │
     ├─ therefore the original seal was removed
     │
     ├─ therefore the bundle was open at some point between S1 and now
     │
     └─ therefore its contents were exposed to whoever was holding it,
        and the holder at that time is known from the custody chain
```

Notice the final step. The mismatch alone says a seal changed. It is the **combination** with the custody chain that names a suspect and a window. This is why the two mechanisms are worth more together than the sum of their parts, and it is the single best thing to demonstrate on `JPR-003`.

D.6 The honest ambiguity

> A mismatch has innocent explanations. A serial mistyped under pressure at 05:40. A damaged seal legitimately replaced with a documented reseal. A courier reading the wrong seal on a multi-bundle consignment.
>
> Mohar **does not decide which**. It records both serials verbatim — `"registered MH-4471-A, read MH-4471-B"` — marks the act as requiring a decision, and stops. A system that concluded "TAMPERING DETECTED" from this evidence would be wrong often enough to be ignored within a month. This is exactly the reasoning behind removing severity labels.

D.7 With the electronic seal lock — the four-way truth table

Once the Phase-5 seal lock exists, there are two independent seal signals: the human-read printed serial, and the device's continuous tamper loop. Combining them is far more informative than either alone.

Printed serial	Tamper loop	Interpretation
Matches	Intact	Clean. Both witnesses agree nothing was opened.
Matches	**Broken**	**The most alarming cell.** The bundle was opened electronically and reclosed under a seal bearing the *original* serial — which means the serial was known in advance and a matching seal was fabricated. This is premeditation, not opportunism.
Differs	Intact	Either a transcription error, or the printed seal was swapped without disturbing the electronic latch. Check the reading against the person and the hour before concluding.
Differs	**Broken**	Overt interference. Both witnesses agree the bundle was opened. Treat the package as compromised.

The reason to build the electronic lock is precisely row two: it is the only configuration that catches a well-prepared attacker, and no purely printed seal can distinguish it from a clean run.

E Key division — S1, S2, S3, S4

> **Naming warning.** The custody *stages* in Part C are also numbered S1–S8. The key *shares* here are S1–S4 and are a completely different thing. In your presentation, say "share S2" and "stage 2" explicitly. Judges notice this collision.

E.1 Why split the key at all

The bundle is encrypted with XChaCha20-Poly1305 under a content key `K`. If `K` exists in one place, that place is a single point of catastrophic failure: whoever holds it can open every paper, early, silently, alone. Splitting `K` converts a secret one party holds into a decision several parties must jointly make.

E.2 Shamir's Secret Sharing — the mechanism

Split `K` into `n = 4` shares with threshold `t = 3`. Any 3 reconstruct `K`; any 2 reveal **nothing whatsoever**.

```
Build a random polynomial of degree t−1 = 2 whose constant term is the secret:

      f(x) = K + a₁x + a₂x²        (a₁, a₂ random, over GF(2⁸))

Hand out points on the curve, never the curve itself:

      S1 = f(1)    S2 = f(2)    S3 = f(3)    S4 = f(4)

3 points determine a unique parabola  ⟶  evaluate f(0) = K
2 points fit INFINITELY many parabolas ⟶  every possible K remains
                                          equally likely
```

> **Why this is stronger than a password split.** Two shares do not "make brute force harder" — they are **information-theoretically** useless. An adversary with two shares and unlimited computing power for a billion years learns exactly nothing about `K`. This is not a hardness assumption; it is a mathematical fact, and it does not degrade with quantum computers.

Implemented in `packages/crypto-core/src/shamir.ts` on the `shamir-secret-sharing` library (Privy) — GF(2^8), zero dependencies, independently audited by Cure53 and Zellic.

E.3 The four holders

Share	Held by	Protected under	Independence
S1	Exam authority	Passphrase-wrapped with Argon2id, stored on our server	Organisation 1
S2	**Nobody, yet**	`tlock` to a future public drand beacon round at `startAt`	Physics, not an organisation
S3	Centre superintendent	WebAuthn platform authenticator — Windows Hello or Android biometric	Organisation 2
S4	Independent observer	WebAuthn platform authenticator on their own phone	Organisation 3

The holder order is fixed in code (`SHARE_HOLDERS`) because share index is meaningful in the audit trail — index 3 always means the superintendent, so a bad share can be attributed to a named party.

E.4 S2 is the keystone

`tlock` (Gailly, Melissaris, Romailler, IACR ePrint 2023/189) encrypts to a *future round* of drand's threshold BLS beacon. The decryption key for that round **does not exist anywhere in the world** until drand's distributed

operators publish it.

```
const round = roundAt(chainInfo, startAt);   // genesis + period * n
const ct    = await tlock.encrypt(shareS2, chainInfo, round);
// always verify chainInfo against the pinned DRAND_CHAIN_HASH first
```

> **Why this is qualitatively different from every other control in the system.** Every other check is a policy decision that some code makes and some code could be made to skip. S2 is not a policy — it is an unforgeable fact about the state of the world. You cannot bribe it, misconfigure it, or patch around it. There is no "admin override" for the passage of time.

E.5 Collusion analysis — the actual security claim

```
To open a paper EARLY you need 3 shares.
Before the beacon publishes, S2 is unobtainable by anyone, anywhere.
So the only early-open set is  { S1, S3, S4 }  - ALL THREE remaining.

   S1  exam authority        ─┐
   S3  centre superintendent ─┼─▶ three separate organisations must
   S4  independent observer  ─┘    collude simultaneously, leaving a
                                   trail in three separate places

After the beacon publishes, S2 is free, so any 2 of {S1,S3,S4} suffice -
which is the intended operating mode: superintendent + observer, the
two-person rule, with the authority share as the recovery path.
```

Note how the threshold does double duty. Before `startAt` it is a strong anti-collusion barrier. After `startAt` it becomes a availability guarantee: any one holder can be ill, unreachable, or have lost their phone, and the exam still runs.

E.6 Commitments — naming the bad share

Raw Shamir has a nasty failure mode worth understanding, because it is the reason our implementation is not just a library call.

> Lagrange interpolation over three points always produces *a* secret. If one share is corrupt or malicious, you get a **wrong key with no error at all** — decryption simply fails later, with no indication of *which* holder supplied bad material. In an exam hall at 08:55 that is an unrecoverable situation with three people accusing each other.

```
every share carries   commitment = sha256(share)

on reconstruction:
  1. check EACH share against its own commitment first
       → mismatch names the holder:
         "share commitment mismatch from: superintendent"
  2. reject duplicate indices
       → "share index 3 (superintendent) supplied more than once"
  3. only then interpolate
  4. verify the recovered secret against a commitment to K itself
```

The result is that a failed unlock produces an attributable, actionable message rather than a mystery.

E.7 The offline fallback — the weakest link, stated plainly

Beacons need network. Many centres have neither reliable connectivity nor power. The fallback: two control-room operators each authenticate and release a replacement for S2, delivered by an operator reading a short code over a phone call.

Every one of these is required:

- Two **distinct** control-room operators authorise.
- Exam authority and independent observer are alerted **synchronously**.
- Rate-limited per exam; exceeding it escalates to a named human.
- `FALLBACK_INVOKED` is written to the ledger **before** the share is released.
- Every invocation reviewed post-exam and the aggregate count **published**.

> This path re-introduces exactly the central-operator risk that S2 exists to remove. It is the weakest link in the design. Measure it and publish it — hiding it is what would actually destroy credibility.

E.8 What is lost without an HSM

With a FIPS 140-2 Level 3 HSM, S1 could never be extracted even under full server compromise. Without one, S1 is a passphrase-wrapped blob on our infrastructure: an attacker who fully owns our servers *and* obtains one more share can open a package early. The threshold split still means we are not a single point of failure — but do not claim HSM-grade custody. If a customer requires it, S1 moves to their HSM with no change to the rest of the design.

F The hardware layer — ESP32, camera, fingerprint

F.1 Three devices, three jobs

Device	MCU	Job	Phase
Room monitor	ESP32-C3 / WROOM-32	Who was in the strongroom, and when. Continuous, non-imaging.	3
Witness station	ESP32-S3 + PSRAM	Who opened the bundle. Event-scoped, biometric + visual.	4 NEW
Seal lock	ESP32-C6	Whether the bundle was physically opened. Continuous tamper loop.	5

F.2 The privacy decision you must make consciously

> **Your own spec currently forbids this.** `docs/06-hardware-spec.md` chose the HLK-LD2410C mmWave sensor specifically because it detects a person without imaging them — the note reads *"no imaging, so no DPDP exposure"* and *"no camera anywhere in the design."* Adding a webcam reverses a deliberate choice. If you present both documents without addressing this, you will be asked about it.

The defensible resolution is **scope**, and it is worth stating as a rule:

	Continuous room surveillance	Event-scoped witness capture ← **do this**
Captures	Everyone entering the room, all day	Two named officials performing one duty
Volume	Hours of video per centre per day	A handful of frames per exam
Legal footprint	Large. Notice, consent, retention policy, DPO obligations at scale	Small. Two consenting officials, narrow purpose, short retention
Defensibility	Hard. "Why are you filming a school corridor?"	Easy. "The two people who opened the sealed papers are photographed at the moment they open them."

So: **the camera fires only on the unlock ceremony**, triggered by the biometric assertions, and never streams. The room monitor keeps its non-imaging sensors and gains no camera. Under India's DPDP Act 2023, biometric and facial data are personal data — purpose limitation and data minimisation are your friends here, and event-scoping gives you both for free.

F.3 Witness station — bill of materials

Component	Part	Approx cost	Purpose
MCU	ESP32-S3-WROOM-1, 8 MB PSRAM	Rs 600	Camera interface, PSRAM for frame buffers, Wi-Fi + BLE
Camera	OV2640 (2 MP) or OV5640 (5 MP)	Rs 350	Still capture at the unlock moment. Not video.
Fingerprint	R307 / AS608 optical, UART	Rs 800	On-module matching; templates never leave the module
Time	DS3231 RTC	Rs 100	Trustworthy timestamps offline
Buffer	microSD module	Rs 100	Write before transmit
Indicator	SSD1306 OLED + buzzer	Rs 200	Two-person state must be visible in the room

Target BOM under Rs 2,200. Higher than the room monitor, but one per centre and only used on exam day.

> **Sensor choice, honestly.** Optical fingerprint modules (R307, AS608) are cheap and adequate, but they are **spoofable** with a lifted print and wood glue or gelatin. A capacitive module (FPC1020 class) resists this considerably better for roughly double the cost. If a judge asks about presentation attacks, the correct answer is "optical in the prototype, capacitive in deployment, and the camera frame is the cross-check" — not "our fingerprint sensor cannot be fooled."

F.4 Why the fingerprint template never leaves the module

```
finger ──► [ R307 module ]
                │  enrols and matches ENTIRELY on-module
                │  stores template in its own flash
                │
                └──► returns only:  matched? + template slot id + score
                                │
           ESP32 ──► HMAC( slot_id ‖ counter ‖ rtc_time ) ──► ledger

The ledger stores:   "template slot 3 matched, score 187, at 08:52:14"
The ledger NEVER stores:   the fingerprint image, or the template
```

> This is both a privacy property and a security one. A database breach cannot leak biometrics that were never in the database, and it is the difference between "we record that the superintendent authenticated" and "we hold a national fingerprint collection." **Do not touch Aadhaar.** Enrol locally, per centre, per exam cycle — Aadhaar-linked biometrics carry statutory restrictions you do not want anywhere near a hackathon prototype.

F.5 How the camera frame becomes evidence

```
1. trigger: a biometric assertion succeeds  (never a timer, never motion)
2. capture: single JPEG frame, ~100 KB
3. hash:    imgHash = SHA256(jpeg bytes)
4. ledger:  the EVENT carries imgHash — the image itself does NOT
5. store:   JPEG written to microSD, uploaded when bandwidth allows
6. verify:  later, anyone can hash the stored image and compare to the
            hash that was committed to the chain at 08:52:14

▸ the chain proves the image is the one captured at that moment
▸ the chain does not become a photo archive
▸ losing the image later does not break the chain
```

This is the same pattern as the custody key: commit the hash, hold the artefact elsewhere. It keeps the ledger small, keeps images out of a system designed for signed facts, and still makes substitution detectable.

F.6 New checks the hardware adds to the engine

The 15 software checks stay exactly as they are. Hardware adds six more, applied only at stage 6 (`unlock`):

#	Check	What it establishes
16	`biometric_primary`	A fingerprint matched the superintendent's enrolled template
17	`biometric_secondary`	A **different** template matched — the observer, not the same finger twice
18	`two_person_copresence`	Both assertions fell inside a short window (e.g. 120 s), so the two people were there together
19	`occupancy_corroborated`	The room monitor's independent headcount is consistent with two people
20	`seal_lock_intact`	The tamper loop was unbroken up to this authorised open
21	`witness_capture`	A frame was captured and its hash committed to the chain

> Every one of these follows the existing rules: **always evaluated**, never short-circuited, and each records *evidence* rather than a verdict — `"slot 3 matched, score 187"`, `"assertions 41 s apart"`, `"monitor reports at least 2 present"`.

F.7 Firmware rules — same for all three devices

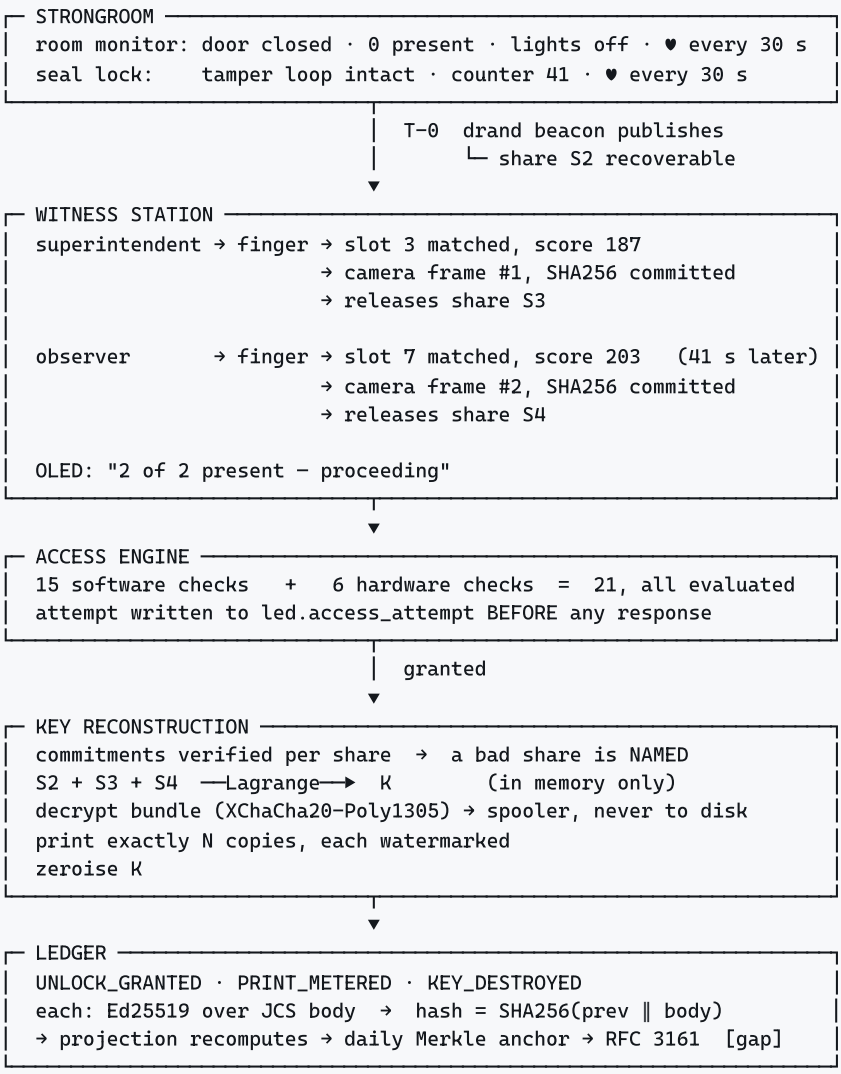
- **Heartbeat every 30 seconds.** Absence of a heartbeat is itself a ledger event. Unplugging the device is not a way to go dark; it is a way to raise an alarm.
- **Write to SD before transmitting.** Cutting power or jamming Wi-Fi must not erase the record.
- **RTC-stamped monotonic sequence numbers**, so gaps are visible rather than silent.
- **HMAC every record** with a per-device key provisioned at flash time.
- **BLE drain fallback** where the school has no usable Wi-Fi — the field app empties the buffer on its next visit.

F.8 Honest limits of the hardware

> **ESP32 flash is readable.** An attacker with physical access and patience can extract the per-device HMAC key and forge records. These devices are **tamper-evident, not tamper-proof**. Their real value is the heartbeat gap and the anomaly log, not cryptographic assurance.
>
> **Optical fingerprint sensors are spoofable** with a lifted print.
>
> **And the point that matters most:** at Hazaribagh the principal was *authorised* to be in that room. Biometrics and occupancy sensing are detective controls against *unauthorised* entry, and close to useless against *authorised* betrayal. Build them; do not let them be the story. The story is the ledger.

G Combined flow map — hardware and software together

G.1 The unlock ceremony, end to end



G.2 What each layer contributes to one question

Take the question an investigator actually asks: *"Who opened package JPR-002, when, and was anyone else present?"*

Layer	Its answer	Could it be faked alone?
Ledger	Device D signed an unlock event at 08:52:14, chained at entry #190	Not without breaking every later hash
Custody keys	A key valid only for that stage and that 6-hour epoch was presented	Not after the epoch passes
Fingerprint	Templates 3 and 7 matched, 41 s apart	Yes — optical sensors are spoofable
Camera	Two frames, hashes committed to the chain at that instant	Only by substituting a face, not a hash
Room monitor	Door opened 08:50, at least 2 present, lights on	Yes — flash is readable
Seal lock	Tamper loop broke at 08:52:30, counter 41 → 42	Yes — with physical access

> **This is the real argument for the hardware.** Individually, every physical sensor is defeatable. Together they are six independent witnesses that must all be corrupted *consistently* — and the ledger, which is the one layer that cannot be quietly rewritten, records what each of them said at the time. An attacker does not need to beat one control; they need to beat all six without contradiction, in a room, in minutes.

H Hardware build roadmap

H.1 Sprint order for the witness station

Step	Deliverable	Proves
1	ESP32-S3 blinks, Wi-Fi joins, RTC reads correct time	Toolchain works
2	R307 enrols a finger and matches it, printing slot + score over serial	Biometric path works standalone
3	OV2640 captures one JPEG to PSRAM and writes it to microSD	Camera path works standalone
4	SHA-256 of the JPEG computed on-device and printed	Commitment is real, not asserted
5	HMAC'd record POSTed to the ledger; appears in the control room	**End-to-end. This is the demo.**
6	Two-person window logic + OLED state + buzzer	The ceremony, not just the sensors
7	Write-to-SD-before-transmit, and heartbeat gap raises an event	Fails closed

> **Stop at step 5 if time is short.** A fingerprint on a breadboard producing a signed row in a live control room is worth more in a demo than a polished enclosure with nothing behind it. Steps 6 and 7 are what make it real; step 5 is what makes it *believable*.

H.2 Integration risks, ranked

Risk	Mitigation
ESP32-S3 camera + Wi-Fi + PSRAM contention causes brownouts and reboots	Capture first, transmit second, never concurrently. Use a supply rated well above 500 mA.
Optical fingerprint fails on dry, worn, or work-hardened fingers — common in exactly this workforce	Enrol three fingers per person. Keep the WebAuthn path as the primary; biometrics **corroborate**, they do not replace.
Camera frames are large; rural upload is slow	Only the hash is on the critical path. The image uploads whenever it can, or is drained over BLE later.
Clock drift makes the two-person window unreliable	DS3231 is the on-device authority; the ledger records device time and server time separately and never corrects one with the other.
Demo hardware fails on stage	Seed a completed ceremony beforehand and have the control-room record open in a second tab. Never let a loose jumper wire cost you the presentation.

H.3 What to say about the hardware in three sentences

> **What it adds.** "The ledger proves what was recorded. The hardware proves a specific human body was physically present at the moment the sealed bundle was opened — and commits a photograph of that moment to the same tamper-evident chain."
>
> **Why it is scoped.** "The camera fires only on the unlock ceremony, never continuously. We photograph two officials performing one duty, not a school corridor. The fingerprint template never leaves the sensor module — we store a match result, not a biometric."
>
> **What it does not do.** "Every one of these sensors is individually defeatable, and we say so in our own spec. Their value is that six independent witnesses must be corrupted consistently, and the one layer that cannot be quietly rewritten is the ledger recording what each of them said."

Mohar — roadmap, flow maps, seal-mismatch theory, key division S1–S4, and the hardware layer. Companion to the main study guide. Software described here is implemented except where marked PLANNED or noted as a gap; all hardware in Parts F–H is design, not built. Source: `docs/learn/mohar-roadmap-and-flows.html`.